authsql 插件深度审查报告

> 版本：0.1.14 | 类型：后端插件 | 代码量：~450 行
>
> 审查日期：2026-03-27

综合评分

维度	评分	简评
功能性	7/10	满足基本需求，但缺少高级功能
安全性	7/10	SQL 参数化合格，refresh token 有哈希
架构设计	6/10	接口实现合理但依赖管理糟糕
接口设计	5/10	错误处理严重不足
错误处理	2/10	大面积静默吞掉异常
可测试性	4/10	无测试用例，强依赖 MySQL
总评	**5/10**	合格的实现，灾难级的错误处理

一、功能性分析

核心功能

authsql 是 authcore 接口的 MySQL 实现，提供三个核心能力：

1. **UserRepository 实现**（`MysqlUserRepository`）
   - 按 ID/email/phone 查询用户
   - 创建用户（密码注册 + 游客注册）
   - 更新密码
   - 密码验证
1. **TokenService 实现**（`MysqlTokenService`）
   - Access Token 签发（JWT）
   - Refresh Token 签发 + 存储（SHA2-256 哈希）
   - Token 吊销（单个 / 用户级别）
   - Token 刷新
1. **外部身份映射**
   - OAuth 提供者 ID → 内部用户 ID 映射
   - 查询 + 创建映射关系

功能缺陷

缺陷	严重度	说明
无软删除	🟡 中	吊销令牌不做物理删除也不做标记
无批量操作	🟡 中	吊销用户所有令牌需逐条 DELETE
无分页查询	🟡 中	用户列表查询无分页支持
无连接池管理	🟠 高	直接使用外部传入的连接池，无重连逻辑

二、安全性分析

✅ 做对的地方

1. **SQL 参数化查询**（所有查询均使用占位符）：

```
final results = await _pool.query(
  'SELECT * FROM wy_user WHERE email = ?',
  [email],
);
```

1. **Refresh Token 数据库哈希存储**：

```
static String _hashToken(String token) {
  return sha256.convert(utf8.encode(token)).toString();
}
// 数据库只存哈希值，即使 DB 泄露也无法直接使用 refresh token
```

⚠️ 安全隐患

问题 *1*：密码未在 *SQL* 层哈希

```
Future<Map<String, dynamic>?> createUserByEmail({
  required String email,
  required String passwordHash,  // ← 已哈希
  String? nickname, String? avatar,
}) async {
  await _pool.query(
    'INSERT INTO wy_user (email, password, ...) VALUES (?, ?, ...)',
    [email, passwordHash, ...],  // ← 存的是哈希值✅
  );
}
```

这部分依赖调用者（authcore）先做哈希。如果 authcore 使用了 InMemoryPasswordHasher（仅存密码长度），则 SQL 层存储的"哈希"毫无意义。SQL 层本身缺乏验证哈希质量的机制。

问题 2：*Token* 吊销使用 *print* 调试

```
Future<void> revokeByUser(int userId) async {
  print('MysqlTokenService revokeByUser: $userId');  // ← 生产代码中的 print
  await _pool.query(
    'DELETE FROM wy_refresh_tokens WHERE user_id = ?',
    [userId],
  );
  print('MysqlTokenService revokeByUser rows affected: ...');
}
```

- `print()` 在生产环境会输出到 stdout，日志格式不可控
- 包含用户 ID 等敏感信息，无日志级别控制
- 应使用统一的 Logger 框架

三、架构设计批判

优点

✅ 职责单一：仅实现 authcore 定义的接口，不越界

✅ **Schema 常量分离**：数据库表/列名提取到 `schema_constants.dart`

✅ 接口实现分离：`MysqlUserRepository` 和 `MysqlTokenService` 各自独立

缺陷

问题 1：依赖管理灾难

```
dependencies:
  auth_core:
    git:
      url: git@xxxx:nicklaus/auth_core.git
      ref: main  # ← 指向 main 分支！

  wxt_mysql:
    git:
      url: https://xxxx/nicklaus/wxt_mysql.git
      ref: main  # ← 又是 main 分支！
```

批判：

- 两个依赖一个用 SSH（`git@`），一个用 HTTPS，风格不统一
- 都指向 `main` 分支，任何上游 push 都会意外影响此包
- 无版本锁定，构建不可重复
- CI/CD 环境需要 SSH key 才能拉取 auth_core

改良方案：

```
dependencies:
  auth_core:
    git:
      url: https://xxxx/nicklaus/auth_core.git
      ref: v0.1.22  # 固定版本标签
  wxt_mysql:
    git:
      url: https://xxxx/nicklaus/wxt_mysql.git
      ref: v0.1.12  # 固定版本标签
```

问题 2：无迁移/建表机制

```
class SchemaConstants {
  static const userTable = 'wy_user';
  static const refreshTokenTable = 'wy_refresh_tokens';
  // ... 列名常量
}
```

定义了表名和列名常量，但完全没有建表 SQL、迁移脚本或 schema 验证。实际建表依赖于外部的 `ephnic.sql` 文件，两者之间无任何关联或一致性保证。列名改了 → SQL 改不改？无人知道。

问题 3：包体积与封装

```
lib/
├── authsql.dart            ← 导出文件
├── schema_constants.dart   ← 数据库常量
├── mysql_token_service.dart
├── mysql_user_repository.dart
└── external_identity_service.dart
```

450 行代码分 5 个文件还算合理，但 `schema_constants.dart` 被导出到外部了。这意味着消费者可以直接拼 SQL 查询 authsql 管理的表，破坏了封装性。

四、接口设计批判

🔴 致命问题：异常被静默吞掉

```
Future<int?> getUserIdByExternal(String provider, String externalId) async {
  try {
    final results = await _pool.query(
      'SELECT user_id FROM wy_external_identity WHERE provider = ? AND
external_id = ?',
      [provider, externalId],
    );
    if (results.isEmpty) return null;
    return results.first['user_id'] as int;
  } catch (_) {  // ← 吞掉所有异常
```

```
    return null;  // ← 数据库挂了？连接超时？返回 null
  }
}

Future<bool> setExternalMapping(...) async {
  try {
    await _pool.query(
      'INSERT INTO wy_external_identity ...',
      [...],
    );
    return true;
  } catch (_) {  // ← 主键冲突？连接断开？网络超时？
    return false;  // ← 全部当作"失败"，不知道为什么失败
  }
}
```

批判：

- `catch (_)` 是代码审查中的绝对红线
- 数据库连接断开 → 返回 `null` → 上层以为"用户不存在" → 创建重复用户
- 主键冲突 → 返回 `false` → 上层以为"写入失败" → 重试 → 又冲突 → 死循环
- 网络超时 → 返回 `null` → 丢失诊断信息 → 排查需要数小时

改良方案：

```
Future<int?> getUserIdByExternal(String provider, String externalId) async {
  try {
    final results = await _pool.query(...);
    if (results.isEmpty) return null;
    return results.first['user_id'] as int;
  } on MySqlException catch (e) {
    _logger.error('查询外部身份失败: provider=$provider, error=${e.message}');
    rethrow;
  }
}
```

问题 2：返回值类型不安全

```
Future<Map<String, dynamic>?> getUserById(int id) async {
  // 返回 Map<String, dynamic>
}
```

所有用户查询返回 `Map<String, dynamic>`，调用者需要记住每个 key 的名称和类型。如果数据库列名修改，编译时不会报错，运行时才崩溃。

改良方案：定义强类型 `UserRecord` DTO：

```
class UserRecord {
  final int id;
  final String? email;
  final String? phone;
  final String? passwordHash;
  final DateTime createdAt;
```

```
  factory UserRecord.fromRow(Map<String, dynamic> row) => UserRecord(
    id: row['id'] as int,
    email: row['email'] as String?,
    // ...
  );
}
```

问题 3：createGuestUser 返回值矛盾

```
Future<Map<String, dynamic>?> createGuestUser({
  String? nickname,
  String? avatar,
}) async {
  await _pool.query('INSERT INTO wy_user ...');
  final results = await _pool.query('SELECT LAST_INSERT_ID() AS id');
  // ...
  return {'id': insertedId, 'nickname': nickname ?? 'Guest', ...};
}
```

- 创建用户 → 返回手动构造的 Map，而非重新查询完整数据
- 这意味着缺失了数据库默认值（如 `created_at` ）
- 与 `getUserById` 返回的结构可能不一致

五、改良建议总结

优先级	改良项	预估工作量
P0	消除所有 `catch (_)` → 使用具体异常类型	1h
P0	替换 `print()` → 使用	0.5h
P1	Git 依赖固定版本标签	0.5h
P1	统一 SSH/HTTPS URL 风格	0.5h
P1	定义 `UserRecord` 强类型返回值	2h
P2	隐藏 `SchemaConstants` 不对外导出	0.5h
P2	创建数据库迁移脚本 + 校验机制	3h
P3	创建用户后重新查询完整记录	1h
P3	增加连接健康检查	1h